Multidimensional Scaling

References can be found in the following textbooks:

- A. Blum and J. Hopcroft and R. Kannan, *Foundations of Data Science.*
- T. F. Cox and M. A. Cox, *Multidimensional Scaling.*

The problem

Assume that we have a method for estimating distances between objects. The goal of Multidimensional Scaling (or more accurately Metric Multidimensional Scaling) is to find an embedding of these objects in a low dimensional Euclidean space so that the measured distances between the embedded objects are similar to the given distances.

Let $x_1, \ldots, x_n$ be the n objects. Suppose δ_{ij} is the estimated distance between x_i and x_j. Given k our goal is to determine the k-dimensional vectors $y_1, \ldots, y_n$ so that $|y_i - y_j| \approx \delta_{ij}$.

Applications of MDS include feature extraction and visualization. The k features extracted from the object x_i are the coordinates of the vector y_i. To visualize the data $x_1, \ldots, x_n$ we can embed it in 1,2, or 3 dimensions and plot the points $y_1, \ldots, y_n$.

STRESS and SSTRESS

There are three classic definitions of the estimation error, that are typically called STRESS, SSTRESS, and STRAIN. The first two can be used even when only few of the distances are estimated. Let Δ be the set of estimated distances.

$$\text{STRESS}(\Delta, y_1, \ldots, y_n) = \sum_{\delta_{ij} \in \Delta} |\delta_{ij}^2 - |y_i - y_j|^2|$$

$$\text{SSTRESS}(\Delta, y_1, \ldots, y_n) = \sum_{\delta_{ij} \in \Delta} (\delta_{ij}^2 - |y_i - y_j|^2)^2$$

Many methods were invented to find $y_1, \ldots, y_n$ so that STRESS or SSTRESS are minimized. The standard approach is to apply numerical optimization techniques. The general view is that the STRESS error is more natural, so that minimizing it gives better results. On the other hand, the minimization of SSTRESS is more efficient. We will not discuss these methods. Software can be found in commercial and freely available statistics packages.

STRAIN

The STRAIN approach is also called classical MDS. Even though the method is straightforward to apply, it is much harder to explain what is being approximated. It is based on the following three key ideas.

1. Assume that the objects $x_1, \ldots, x_n$ are vectors. Given the true distances δ_{ij} between all pairs of *vectors*, it is possible to compute their Gram matrix. The Gram matrix G has the value of $x_i^T x_j$ at matrix location ij.

2. Given the Gram matrix G of n vectors it is possible to determine exactly an embedding of these vectors in $\mathbb{R}^k$, whenever such embedding exists.

3. Given the Gram matrix G of n vectors, and a number $k \leq n$, it is possible (and very easy) to determine the Gram matrix G_k most similar to G that will allow exact embedding in $\mathbb{R}^k$.

These three key ideas form the STRAIN algorithm.

Input: all approximate distances δ_{ij}, and a (typically small) value for k.

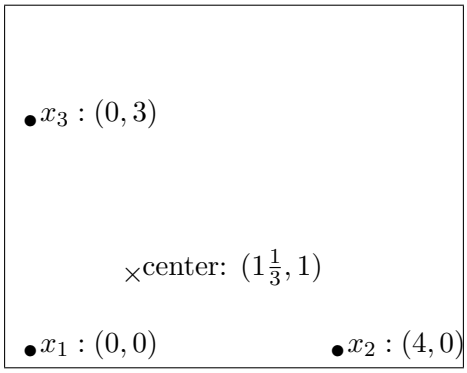
Output: the vectors $y_1, \ldots, y_n$ in $\mathbb{R}^k$.

1. Compute the Gram matrix G assuming that the distances are accurate.

2. Compute the Gram matrix G_k.

3. Compute the vectors $y_1, \ldots, y_n$ form G_k.

In describing the algorithm details we will follow a simple example of a right triangle. Consider a right triangle with corners at the following coordinates:



The distance matrix D of these 2D points is given by:

$$D = \begin{pmatrix} 0 & 4 & 3 \\ 4 & 0 & 5 \\ 3 & 5 & 0 \end{pmatrix}$$

Step 1: Compute the Gram matrix G from D

Since we assume here that the distances are accurate we have:

$$\delta_{ij}^2 = |x_i - x_j|^2 = |x_i|^2 + |x_j|^2 - 2x_i^T x_j \tag{1}$$

This shows that $x_i^T x_j$ can be computed if we can compute the norms $|x_i|^2, |x_j|^2$. Without loss of generality we can assume that the location of $(0, 0)$ is at the "center" of the points. This means:

$$\sum_{i=1}^{n} x_i = 0 \tag{2}$$

The following values can be computed easily from the distance matrix:

$$\begin{aligned} a_{ij} &= \delta_{ij}^2 \\ a_i &= \sum_{j=1}^{n} a_{ij} \\ a &= \sum_{i=1}^{n} a_i \end{aligned} \tag{3}$$

Summing up Equation (1) over all j values we get:

$$a_i = n|x_i|^2 + \sum_{j=1}^{n} |x_j|^2 \tag{4}$$

Summing up Equation (4) over all i values we get:

$$a = n\sum_{i=1}^{n} |x_i|^2 + n\sum_{j=1}^{n} |x_j|^2 = 2n\sum_{i=1}^{n} |x_i|^2 \tag{5}$$

From (4),(5) we have:

$$\sum_{i=1}^{n} |x_i|^2 = a/2n, \quad |x_i|^2 = a_i/n - a/2n^2$$

Substituting in (1) we have:

$$x_i^T x_j = -a_{ij}/2 + a_i/2n + a_j/2n - a/2n^2 \tag{6}$$

This is an explicit formula for the Gram matrix. In our example we have:

$$G = \frac{1}{9}\begin{pmatrix} 25 & -23 & -2 \\ -23 & 73 & -50 \\ -2 & -50 & 52 \end{pmatrix} \approx \begin{pmatrix} 2.77778 & -2.55556 & -0.222222 \\ -2.55556 & 8.11111 & -5.55556 \\ -0.222222 & -5.55556 & 5.77778 \end{pmatrix}$$

Steps 2 and 3: Compute the vectors from the Gram matrix

Consider the matrix $X = (x_1, \ldots, x_n)$. It is related to the Gram matrix G by:

$$G = X^T X$$

Therefore, X can be computed from the EVD of G as follows:

$$G = V\Sigma V^T = X^T X \Rightarrow X = \Sigma^{1/2} V^T$$

Observe that the matrix product on the right is simply the scaling of the ith eigenvector by a factor of $\sqrt{\lambda_i}$. This gives the following algorithm for constructing X from G:

1 For $i = 1, \ldots n$ compute $u_i = \sqrt{\lambda_i} v_i$.

2. $X = (u_1, \ldots, u_n)^T$

How can we tell if it is possible to obtain embedding in dimension $k < n$? If this is true then the degree of G is at most k. Specifically, there will be only k nonzero (positive) eigenvalues. If G has more than k positive eigenvalues we can search for a positive semidefinite and symmetric matrix G_k that is most similar to G and has only k positive eigenvalues. It can be shown that this matrix is obtained from G by the following process. If G has negative eigenvalues replace them with 0. After this replace all but the k largest eigenvalues with 0. This gives the following algorithm for parts 2,3:

Input: an approximation for the Gram matrix G obtained in Step 1, and a value of $k \leq n$.

Output: the k dimensional vectors $y_1, \ldots, y_n$.

1. Compute the EVD $G = V\Sigma V^T$.

2. Take the k largest eigenvalues in Σ and replace all negative values (if there are any) with 0.

3. For $i = 1, \dots k$ compute $u_i = \sqrt{\lambda_i} v_i$.

4. $Y = (u_1, \dots, u_k)^T$.

Applying this to our example:

$$G = V \text{ Diag}(l) \ V^T$$

$$V = \begin{pmatrix} 0.18278456661844997 & 0.7957743412585183 & -0.5773499805150057 \\ -0.7805526882430602 & -0.23959097272276272 & -0.5773505578643635 \\ 0.5977686032928097 & -0.5561828508542245 & -0.577350269189364 \end{pmatrix}$$

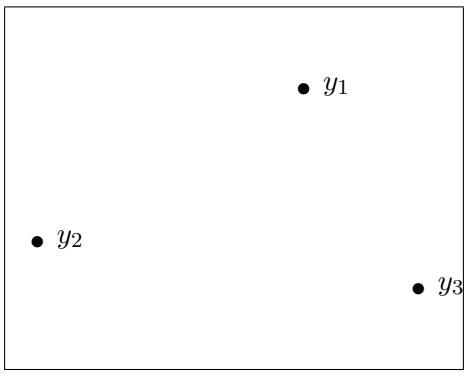
$$l = \begin{pmatrix} 12.964153652060633 & 3.702521014607366 & -4.6666679991744 * 10^{-}6 \end{pmatrix}$$

The last eigenvalue is nonzero because of round-off errors. In exact arithmetic it is supposed to be 0. This shows that the data (3 points) can be embedded exactly in 2D, but only approximately in 1D.
To compute the 2D embedding:

$$\begin{aligned} u_1 &= \sqrt{12.964153652060633}\ (0.18278456661844997, -0.7805526882430602, 0.5977686032928097) \\ &\approx (0.65813, -2.81044, 2.15231) \\ u_2 &\approx (1.53122, -0.461019, -1.0702) \end{aligned}$$

This gives:

$$y_1 = \begin{pmatrix} 0.65813 \\ 1.53122 \end{pmatrix} \quad y_2 = \begin{pmatrix} -2.81044 \\ -0.461019 \end{pmatrix} \quad y_3 = \begin{pmatrix} 2.15231 \\ -1.0702 \end{pmatrix}$$



This is the same as the original configuration except for translation and rotation.
To compute the 1D embedding:

$$u_1 \approx (0.65813, -2.81044, 2.15231)$$

This gives:

$$y_1 = 0.65813, \quad y_2 = -2.81044, \quad y_3 = 2.15231$$

This preserves a lot of the original information. The two farthest points are y_2 and y_3, and y_1 is closer to y_3 than to y_2.

Things to keep in mind

- Consider the special case where the original objects $x_1, \ldots, x_n$ are given as vectors in $\mathbb{R}^{k_1}$ and the classical MDS is applied in order to reduce the dimensionality to $\mathbb{R}^{k_2}$, with $k_2 \leq k_1$. Here we may compute $\delta_{ij}^2 = |x_i - x_j|^2$. In this special case it can be shown that the dimensionality reduction obtained by MDS is identical to the dimensionality reduction obtained by the PCA technique. MDS is useful when the distances are not computed as above.

- It is important to observe that the input to the algorithm is the values of δ_{ij}^2 and not δ_{ij}. Typically one tries to use dissimilarities as measure of distances. You have to decide if the dissimilarities are a related to δ_{ij} or to δ_{ij}^2.